DISEÑO DE APLICACIONES WEB

MEMORIA DE IMPLANTACIÓN Y DESPLIEGUE: FRUTWEB

CÓDIGO DEL PROYECTO: FRUTWEB

CICLO DAW

FECHA FEBRERO 2026

Autor: Mario Novo Calvo

Tutora: María Del Carmen Mateos Batista

REPOSITORIO: https://github.com/Arbak-canarias/FRUTIWEB

Versión	Fecha	Autor	Descripción de los cambios
1.0	19/02/2026	Mario Novo Calvo	Creación inicial del documento despliegue .
1.1	20/02/2026	Mario Novo Calvo	Corrección de rutas estáticas y solución de errores de BD.

ÍNDICE

Índice

1. INTRODUCCIÓN....3
2. REQUISITOS PREVIOS DEL SISTEMA....3
2.1. Servidor de Aplicaciones....3
2.2. Servidor de Base de Datos....3
2.3. Git para Windows....4
3. CONFIGURACIÓN DEL ENTORNO....4
2.1. Archivo de configuración (.env)....5
4. DESPLIEGUE DE LA BASE DE DATOS....5
4.1. Instrucciones de carga....5
5. INSTALACIÓN Y PUESTA EN MARCHA....6
6. VERIFICACIÓN Y PUNTOS DE ACCESO....6
7. CONTROL DE VERSIONES Y TRAZABILIDAD....6
8. CONCLUSIÓN....7
7.ERRORES SUBSANADOS PARA PODER DESPLEGAR....8

1. INTRODUCCIÓN

El presente documento describe el proceso de implantación y despliegue de la aplicación web **FRUTWEB**, una plataforma de gestión de frutería online basada en arquitectura cliente-servidor. Se detallan los requisitos técnicos, la configuración del entorno, la carga de la base de datos y los pasos necesarios para la puesta en marcha del sistema.

2. REQUISITOS PREVIOS DEL SISTEMA

Para garantizar el correcto funcionamiento de la aplicación en un entorno Windows 11, se requiere disponer del siguiente software:

2.1. Servidor de Aplicaciones

- **Software:** Node.js
- **Versión mínima: v22.13.0 LTS**.
- **Justificación:** Es el "motor" que hace que el archivo server.js funcione. Sin Node.js, el comando npm install o npm start no funcionará y te dará error de "comando no encontrado". Se requiere la versión Long Term Support para asegurar la compatibilidad con las librerías de cifrado (bcrypt) y gestión de tokens (JWT).
- **Gestor de paquetes:** npm v10.9.0 o superior.

2.2. Servidor de Base de Datos

- **Software:** MariaDB
- **Versión mínima: v12.2.2**.
- **Configuración:** El servicio debe estar escuchando en el puerto estándar **3306**.
- **Motor de almacenamiento:** InnoDB (para soporte de claves foráneas e integridad referencial).
- **Justificación:** Soporte de claves foráneas e integridad referencial.

2.3. Git para Windows.

- **Versión mínima:** v2.53.0 Se utilizará la consola **Git Bash** para la clonación y gestión del repositorio.

Nota: Si tras la instalación el comando `node -v` no reconoce la versión correcta, reinicie su terminal Git Bash para actualizar las variables de entorno.

3. CONFIGURACIÓN DEL ENTORNO

Antes de iniciar la aplicación, es necesario configurar las variables de entorno mediante un archivo ***.env.***

3.1. Archivo de configuración (.env)

En la raíz del proyecto, se debe crear a .env y parametrizar los siguientes valores:

Parámetro	**Valor por defecto**	**Descripción**
PORT	3000	Puerto de escucha de la aplicación web.
DB_HOST	localhost	Dirección del servidor de MariaDB.
DB_USER	root	Usuario con privilegios en la base de datos.
DB_PASSWORD	(vacío)	Contraseña del usuario de la base de datos.
DB_NAME	frutweb_db	Nombre de la base de datos.
JWT_SECRET	frutweb_secret_2026	Clave de cifrado de tokens.

Como el repositorio GitHub no recomienda compartirlo por eso debemos crearlo manualmente en la raíz de la carpeta `FRUTIWEB` con los datos de conexión a tu base de datos local con el nombre exacto **.env** (asegúrate de que no se guarde como .env.txt)

Si lo has guardado como txt debes renombrar desde la terminal escribe este comando y pulsa enter

mv .env.txt .env

4. DESPLIEGUE DE LA BASE DE DATOS

Para la puesta en marcha del modelo de datos, se debe utilizar el script proporcionado en la entrega.

4.1. Instrucciones de carga

1. Acceder a la consola de MariaDB o herramienta de gestión (HeidiSQL/DBeaver).
2. Crear el esquema:

   SQL

   ```
   CREATE DATABASE frutweb_db
   CHARACTER SET utf8mb4
   COLLATE utf8mb4_unicode_ci;
   ```

3. Importar el archivo : **frutweb_db.sql** está en la carpeta BBDD

mysql -u root -p frutweb_db < frutweb_db.sql

Si no te ves capaz, entonces, botón derecho - crear nuevo → Base de datos → frutweb_db

Una vez creada, la importas desde la carpeta FRUTWEB estará BBDD y te debe quedar algo así.

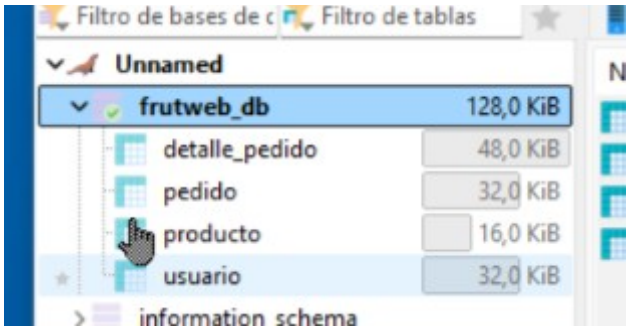


Figura 1: Base de Datos funcionando.

5. INSTALACIÓN Y PUESTA EN MARCHA

Siga estos pasos en orden estricto para desplegar la aplicación en Windows:

Paso 1: Instalación de dependencias

Siga este orden estricto en la terminal Git Bash:

https://git-scm.com/install/windows

1. Clonación: git clone https://github.com/Arbak-canarias/FRUTIWEB.git

   cd FRUTIWEB

2. Dependencias: npm install (Instala Express, Mysql2, Bcrypt, etc.).

¡OJO si no funciona, asegúrate de que se cargue la versión de node.js

node -v si no te sale la versión v22.22.0 sal de la consola de gui, asegúrate de que esté instalado, entra al bash de git y prueba otra vez.

Figura 2: Revisión si Node.js Funciona

Ahora npm install

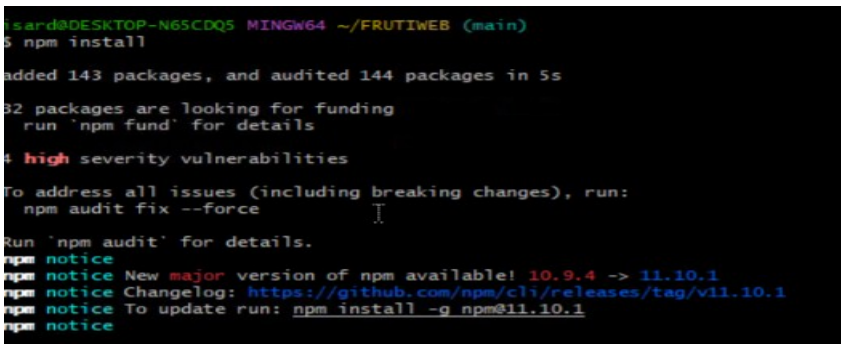


Figura 3: Instalación npm

Paso 2: Verificación de puertos

Asegúrese de que el puerto **3000** no está siendo utilizado por otra aplicación:

netstat -ano | findstr :3000

Paso 3: Inicio de la aplicación

Para iniciar el servidor en modo producción, ejecute:

npm start

6. VERIFICACIÓN Y PUNTOS DE ACCESO

Una vez iniciado el servicio, la aplicación será accesible a través de las siguientes URLs:

- **Interfaz de Local:** http://localhost:3000/index.html
- **Panel de Administración y Cliente:** Acceso restringido mediante login desde la interfaz principal.

7.ERRORES SUBSANADOS PARA PODER DESPLEGAR

Durante el despliegue se resolvieron los siguientes puntos críticos:

- **Rutas Estáticas:** El servidor estaba configurado originalmente con app.use(express.static("public")), lo que impedía localizar los archivos. Se ajustó el código en server.js para que el servidor descargue correctamente los archivos desde la raíz o la carpeta src según corresponda.
- **Compatibilidad SQL:** Se detectó un error de sintaxis en el archivo SQL relacionado con los disparadores (Triggers) y el comando DELIMITER. Para asegurar el despliegue, se limpió el código de triggers no compatibles con la versión específica de MariaDB utilizada.

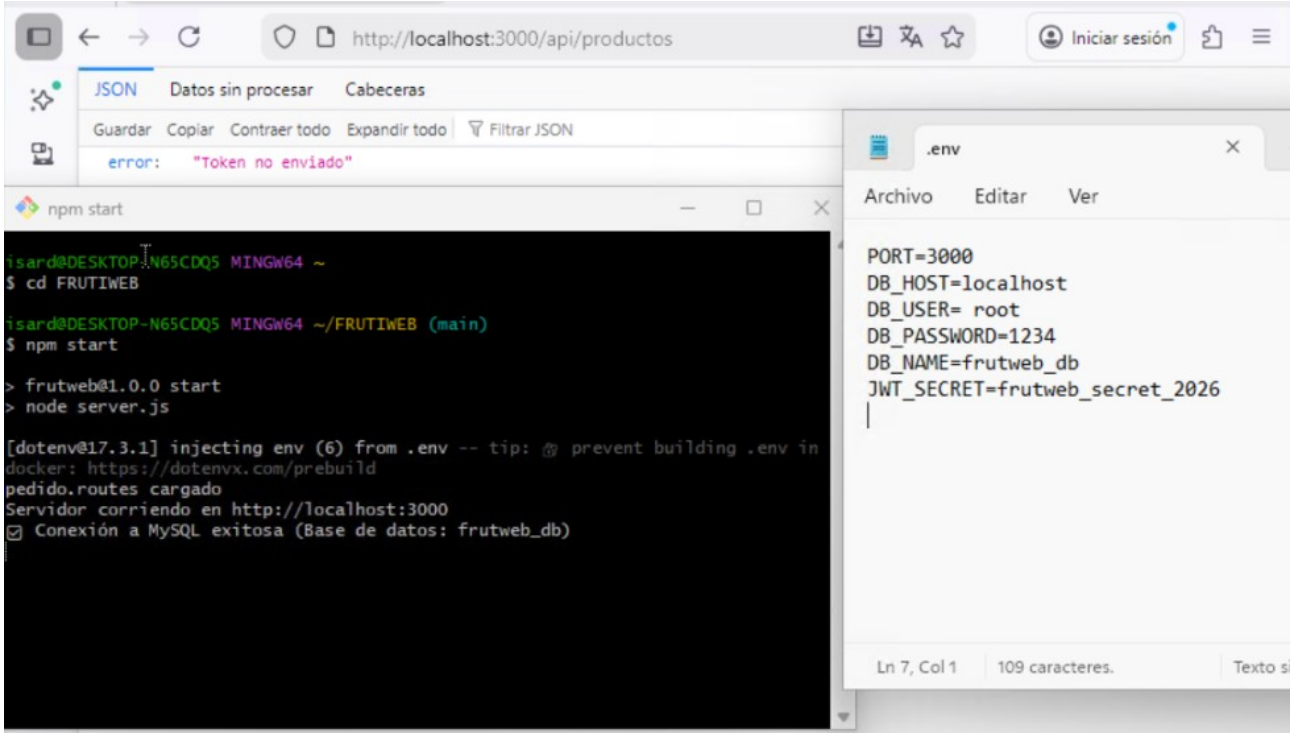


Figura 4: Versión que funcionaba pero no tiene acceso a la web

Funcionar, funciona pero no se ven las webs luego se ha ido a server.js y se ha modificado ya que al tener:

app.use(express.static("public")); no funcionaba,por tanto, se cambio el código a este otro:

```
import express from "express";
import cors from "cors";
import path from "path";
import { fileURLToPath } from "url";

import authRoutes from "./routes/auth.routes.js";
import productoRoutes from "./routes/producto.routes.js";
import usuarioRoutes from "./routes/usuario.routes.js";
import pedidoRoutes from "./routes/pedido.routes.js";

const app = express();

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

app.use(cors({
  origin: ["http://localhost:5500","http://127.0.0.1:5500"],
  credentials: false,
}));

app.use(express.json());

// APIs
app.use("/api/auth", authRoutes);
app.use("/api/productos", productoRoutes);
app.use("/api/usuarios", usuarioRoutes);
app.use("/api/pedidos", pedidoRoutes);

// SERVIR FRONTEND
app.use(express.static(path.join(__dirname, "..")));

// Ruta raiz
app.get("/", (req, res) => {
  res.sendFile(path.join(__dirname, "..", "index.html"));
});

export default app;
```

Figura 5: Cambio server.js

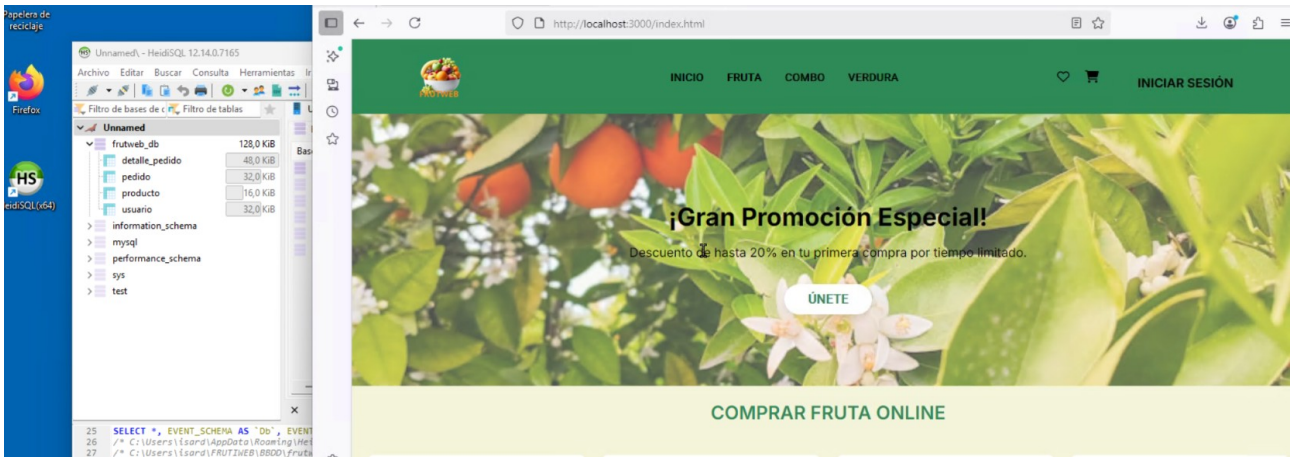
Además se tuvo un problema de importación con la base de datos que obligaba a quitar el código siguiente:

```
-- DELIMITER $$ CREATE TRIGGER
detalle_pedido_estado
BEFORE INSERT ON detalle_pedido
FOR EACH ROW BEGIN DECLARE
estado_padre ENUM('pendiente','enviado','entregado');
SELECT estado INTO estado_padre FROM pedido WHERE id_pedido = NEW.id_pedido;
SET NEW.estado = estado_padre; END $$
DELIMITER ;
```

Ya que no es compatible con la versión desplegada una vez realizado todo eso se consigue desplegar y que sea funcional.

8. CONTROL DE VERSIONES Y TRAZABILIDAD

Enlace al repositorio remoto: https://github.com/Arbak-canarias/FRUTIWEB

9. CONCLUSIÓN

La aplicación FRUTWEB queda completamente operativa tras seguir el procedimiento descrito, permitiendo la gestión de productos, pedidos y usuarios mediante una arquitectura robusta basada en Node.js y MariaDB.